Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра электронных вычислительных средств
Дисциплина «Программирование на языках высокого уровня»

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА
к курсовому проекту
на тему:
«РАЗРАБОТКА ПРОГРАММЫ ТУРАГЕНСТВА»

Выполнил студент группы 450701
Синица Артём Сергеевич

_____
(подпись студента)

Курсовой проект представлен на проверку ___.____.2025

_____
(подпись студента)

Минск 2025

СОДЕРЖАНИЕ

Введение……..... 5
Обзор используемых основных технологий..... 6
1.1 Основы хранения и обработки данных 6
1.2 Алгоритмы обработки данных и методы взаимодействия 7
1.3 Использование хэш-сумм для проверки целостности данных..... 8
1.4 Реализация программной системы 9
1.5 Описание инструментов и технологий 10
2 Разработка алгоритмов 12
2.1 Составление алгоритмов на основе запросов..... 12
2.2 Рассмотрение выбранных алгоритмов 13
3 Програмная реализация 16
4 Результаты и анализ 21
Заключение 23
Список используемых источников..... 24
Приложение А..... 25
Приложение Б 32
Приложение В……...32
Приложение Г……...32

ВВЕДЕНИЕ

Современное развитие информационных технологий оказывает значительное влияние на все сферы человеческой деятельности, включая туризм. Туристические агентства ежедневно сталкиваются с необходимостью обработки большого объёма информации — данных о клиентах, турах, маршрутах, бронированиях и оплатах.

Ручная обработка этих данных занимает значительное время и сопровождается риском ошибок, что снижает эффективность работы агентства. Поэтому внедрение программных средств автоматизации становится важной задачей для повышения производительности труда, точности учёта и качества обслуживания клиентов.

Цель курсового проекта — разработать программное обеспечение, предназначенное для автоматизации деятельности туристического агентства. Программа должна обеспечивать хранение, обработку и вывод информации о клиентах, турах и бронированиях.

Основные задачи:

- Изучить основы объектно-ориентированного программирования на языке C++.
- Рассмотреть особенности работы с файлами и потоками данных в языке C++.
- Проанализировать предметную область и определить функциональные требования к системе.
- Спроектировать архитектуру программы и описать алгоритмы работы.
- Реализовать программу, проверить её корректность и провести тестирование.

Актуальность темы заключается в том, что разработанная программа упрощает управление туристическими услугами и клиентской базой. Система позволяет ускорить обработку заказов, минимизировать вероятность ошибок и облегчить взаимодействие между клиентом и оператором. В дальнейшем программа может быть расширена до сетевого приложения или интегрирована с базой данных.

1 ОБЗОР ИСПОЛЬЗУЕМЫХ ОСНОВНЫХ ТЕХНОЛОГИЙ

1.1 Основы хранения и обработки данных

При проектировании программного обеспечения для туристических агентств ключевое значение имеет надёжная организация хранения и обработки данных. Основная цель таких систем — обеспечить удобный доступ к информации о клиентах, турах и бронированиях, исключив ручную обработку и снизив вероятность ошибок [1].

В отличие от распределённых корпоративных решений, локальные приложения не требуют установки и обслуживания серверных баз данных. В рамках учебного проекта целесообразно использовать файловое хранение информации, что упрощает реализацию и делает программу независимой от сетевого подключения.

Каждая сущность системы (тур, клиент, заказ) хранится в отдельном текстовом файле.
Структура хранения данных включает:

- accounts.txt — учётные записи пользователей с их ролями;
- tours.txt — каталог доступных туров (страна, цена, длительность, описание);
- bookings.txt — записи о бронированиях пользователей.

Файлы создаются автоматически при первом запуске программы, а их структура организована в формате строк с разделителями, аналогичными CSV. Каждая строка соответствует отдельной записи, что позволяет выполнять добавление, редактирование и удаление данных стандартными средствами языка C++.

Основные принципы, реализованные в системе хранения:

–Простота и наглядность — данные можно просматривать и редактировать вручную;

–Устойчивость к сбоям — программа создаёт резервные копии при записи;

–Оптимальность операций ввода-вывода — используется построчная обработка данных с помощью std::fstream;

–Минимальная зависимость от внешних библиотек — применяется только стандартная библиотека STL.

Такой подход обеспечивает прозрачность работы программы, лёгкость отладки и надёжность при выполнении типичных пользовательских операций (поиск, фильтрация, добавление записей).

1.2 Алгоритмы обработки данных и методы взаимодействия

Для эффективной работы приложения разработаны алгоритмы, которые обеспечивают корректную обработку пользовательских запросов и поддерживают внутреннюю целостность данных [2].

Основные реализованные алгоритмы

Инициализация системы — при запуске программы выполняется проверка наличия файлов данных. При их отсутствии создаются пустые файлы по умолчанию.

Чтение данных — строки считываются с помощью функции std::getline(), затем разбиваются на поля с помощью std::stringstream.

Поиск записей — используется линейный поиск по вектору структур std::vector, что обеспечивает достаточную скорость при небольших объёмах данных.

Сортировка — применяется стандартный алгоритм std::sort() для упорядочивания туров по цене, стране или названию.

Сохранение изменений — при изменении информации данные перезаписываются в файл.

Проверка корректности данных — осуществляется контроль диапазона цен, длительности, а также валидация текстовых полей.

Дополнительно стоит рассмотреть принципы работы алгоритмов хеширования, которые лежат в основе контроля целостности данных. В современных программных системах применяются различные алгоритмы хеширования, такие как FNV-1a и CRC32. Они выполняют преобразование входных данных произвольной длины в уникальное числовое значение фиксированного размера.

FNV-1a основан на операциях умножения на простое число и применении операции XOR, что обеспечивает высокую скорость и равномерное распределение результатов. CRC32 изначально разработан для контроля целостности данных и эффективно обнаруживает ошибки при чтении или записи файлов.

В реализованной программе используется упрощённый вариант CRC-алгоритма, включающий следующие этапы:
–инициализация хеша начальным значением;
–последовательное применение операций XOR и умножения на простое число;
–дополнительные битовые сдвиги для повышения равномерности распределения;
– финальное вычисление контрольной суммы.

Преимущества данного подхода:
– высокая скорость вычислений;
– устойчивость к случайным совпадениям (коллизиям);
– простота реализации при сохранении надёжности;
– низкие требования к вычислительным ресурсам.

Таким образом, использование простых алгоритмов хеширования, таких как CRC32 и FNV-1a, позволяет реализовать эффективный контроль целостности данных в учебных программных системах, не прибегая к сложным криптографическим методам [6].

1.3 Использование хэш-сумм для проверки целостности данных

Для контроля корректности работы программы и защиты от случайного повреждения файлов реализован механизм проверки хэш-суммы данных**.** Хэш-сумма представляет собой уникальную числовую подпись, полученную в результате обработки содержимого файла по определённому алгоритму [3]. Она позволяет убедиться, что информация в файле не была случайно изменена или повреждена при записи.

В программе применяется упрощённая модификация алгоритма CRC32, основанная сдвигах.
Данный метод отлично подходит для учебных проектов, где требуется контроль целостности данных без сложных механизмов защиты.

Проверка выполняется следующим образом:

1. Перед сохранением файла вычисляется его хэш-сумма и сохраняется в отдельном служебном файле (checksum.txt).
2. При следующем запуске программы вычисляется текущий хэш-файл и сравнивается с сохранённым значением.
3. В случае несовпадения пользователю выводится предупреждение о возможном повреждении данных.

Преимущества применения хэш-сумм:

–повышается надёжность хранения данных;

–программа способна обнаруживать случайные ошибки записи;

–реализуется простая форма контроля целостности.

Пример вычисления контрольной суммы (упрощённый фрагмент):

```
unsigned int calculateChecksum(const std::string& filename) {
   std::ifstream file(filename);
   unsigned int hash = 0;
   char c;
   while (file.get(c)) {
      hash = (hash * 31) ^ c;  // побитовые операции для вычисления хэша
   }
   return hash;
}
```

Таким образом, применение хэш-сумм в программе повышает устойчивость к ошибкам при чтении и записи данных и является дополнительным средством контроля корректности файлов.

1.4 Реализация программной системы

Реализация программы выполнена на языке C++ с использованием объектно-ориентированного подхода и стандартных библиотек STL. Архитектура программы модульная, что позволяет изолировать отдельные части функционала и упростить сопровождение проекта.

Основные модули программы:

- Модуль пользователей. Реализует регистрацию, авторизацию, удаление и просмотр учётных записей. Данные сохраняются в файле accounts.txt.
- Модуль туров. Содержит функции добавления, редактирования, удаления и поиска туров. Каждый тур хранит информацию о названии, стране, цене, длительности и количестве мест;
- Модуль бронирования. Отвечает за создание, просмотр и удаление бронирований. Каждая запись содержит имя пользователя, идентификатор тура и дату бронирования;

- Модуль интерфейса. Обеспечивает взаимодействие с пользователем через консольное меню. Меню позволяет выбрать действие и направляет выполнение программы в нужный модуль;
- Модуль администратора. Предназначен для управления пользователями и турами, а также для просмотра общей статистики бронирований.

Особое внимание уделено обработке ошибок и устойчивости к некорректным данным**.** При вводе неверных значений (например, букв вместо чисел) программа не завершается аварийно, а сообщает об ошибке и предлагает повторить ввод.

1.5 Описание инструментов и технологий

Язык программирования — C++

C++ выбран как основной язык разработки благодаря высокой производительности, поддержке объектно-ориентированного проектирования и наличию стандартной библиотеки STL [4]. Язык позволяет эффективно работать с памятью, файлами, что делает его оптимальным выбором для локальных учебных приложений.

Среда разработки — Microsoft Visual Studio Code

Для реализации и отладки программы использовалась среда Microsoft Visual Studio Code обеспечивает встроенные инструменты компиляции, отладки, профилирования и тестирования, что ускоряет процесс разработки..

Стандартная библиотека STL

STL (Standard Template Library) предоставляет готовые контейнеры и алгоритмы, которые существенно упрощают реализацию программы. В данной работе применяются:

–std::vector — для хранения туров, пользователей и бронирований;

–std::string — для строковых операций;

–std::fstream — для работы с файлами;

–std::algorithm — для сортировки и поиска;

–std::map и std::pair — для ассоциативных структур данных.

Система хранения данных — текстовые файлы

Хранение информации в текстовых файлах (accounts.txt, tours.txt, bookings.txt) обеспечивает прозрачность и лёгкость проверки данных. Формат хранения близок к CSV, что делает возможным анализ данных даже без запуска программы.

Средства контроля версий — Git и GitHub

Для ведения исходного кода и контроля изменений применялась система Git. Хранение проекта на GitHub обеспечивает резервное копирование и возможность коллективной работы над программой [5].

Обоснование выбора технологий

Выбранные технологии и инструменты обеспечивают:

–простоту реализации и сопровождения программы;

–надёжность хранения и обработки данных;

–кроссплатформенность и независимость от сторонних сервисов;

–эффективность выполнения и минимальные требования к оборудованию.

Таким образом, использование языка C++, стандартной библиотеки STL, текстовых файлов и современных средств разработки позволило создать стабильную, понятную и функциональную систему автоматизации работы турагентства.

2 РАЗРАБОТКА АЛГОРИТМОВ

2.1 Составление алгоритмов на основе запросов

Разработка программы началась с анализа требований и составления набора алгоритмов, обеспечивающих обработку типичных пользовательских запросов[1].

Каждая функция системы — это результат выполнения определённого алгоритма, представленного в виде последовательности шагов: ввод данных, обработка информации и вывод результата.

Для упрощения и наглядности проект реализован в виде консольного приложения, где пользователь взаимодействует с системой через меню. Каждому пункту меню соответствует определённый алгоритм, который обрабатывает запрос и возвращает ответ в читаемой форме.

Основные запросы пользователей:

–просмотр списка туров;

–регистрация и авторизация;

–добавление нового тура (для администратора);

–бронирование тура;

–просмотр истории бронирований.

Каждый запрос описывается в виде алгоритма, включающего:

1. Ввод данных (например, имя пользователя, страна тура, дата).
2. Проверка корректности введённой информации.
3. Обработка запроса — поиск, фильтрация, запись или удаление данных.
4. Вывод результата — подтверждение операции, сообщение об ошибке или итоговый список.

Пример — алгоритм бронирования тура:

1. Пользователь выбирает пункт меню «Забронировать тур».
2. Программа открывает файл tours.txt и выводит список доступных направлений.
3. Пользователь вводит номер нужного тура.
4. Проверяется наличие мест.
5. При успешной проверке создаётся запись в bookings.txt с данными пользователя и текущей датой.

6. Выводится сообщение о подтверждении бронирования.

Такой подход делает систему прозрачной: каждый запрос обрабатывается по строгому алгоритму, что упрощает отладку и добавление нового функционала.

2.2 Рассмотрение выбранных алгоритмов

На этапе реализации были выбраны алгоритмы, обеспечивающие стабильность, предсказуемость результатов и простоту сопровождения кода [3]. Все алгоритмы основаны на классических конструкциях языка C++: циклах (for, while), условных операторах (if, switch) и функциях обработки строк и файлов.

Алгоритм регистрации пользователя

Цель — создание новой учётной записи в системе. Шаги выполнения:

1. Ввод имени пользователя и пароля.
2. Проверка, существует ли пользователь с таким именем в accounts.txt.
3. Если запись отсутствует — добавление новой строки с логином и паролем.
4. Сохранение изменений и вывод подтверждения.

Для предотвращения ошибок используется проверка корректности ввода (непустые строки, допустимые символы).

Алгоритм авторизации

Назначение**:** предоставление доступа к функциям системы на основе роли пользователя. Программа выполняет сравнение введённых данных с записями в accounts.txt. При успешном совпадении определяется тип пользователя — клиент или администратор. Для защиты от случайных сбоев при чтении применяется обработка исключений (try / catch) и проверка состояния потока.

Алгоритм добавления тура

Шаги:

1. Администратор вводит параметры нового тура (страна, цена, длительность).
2. Программа проверяет, что все поля заполнены корректно.

3. Вектор std::vector<Tour> дополняется новой структурой Tour.
4. Список туров пересохраняется в tours.txt.
5. Отображается обновлённая таблица туров.

Для ускорения сортировки туров применяется функция std::sort() с пользовательским компаратором, который сравнивает по цене или стране.

Алгоритм бронирования тура

Цель**:** запись информации о выбранном туре в файл бронирований. После выбора пользователем тура программа:

1. Проверяет наличие свободных мест.
2. Формирует запись вида: Имя_пользователя; Название_тура; Дата_бронирования;
3. Сохраняет данные в bookings.txt;
4. Обновляет количество оставшихся мест в tours.txt.

Для упрощения поиска бронирований используется линейный перебор (for (auto &item : bookings)), что обеспечивает достаточную скорость при небольшом объёме данных [4].

Алгоритм сортировки и фильтрации

Для удобства работы пользователя реализованы функции сортировки туров по цене, названию и стране. Используется стандартный алгоритм std::sort() из библиотеки <algorithm>. Фильтрация выполняется через цикл for и сравнение строк std::string::find(), что позволяет быстро находить нужные туры.

Алгоритм проверки целостности данных (хэш-сумма)

Для контроля корректности файлов при запуске программы вычисляется хэш-сумма содержимого (CRC32-подобный алгоритм). Если значение хэша не совпадает с сохранённым, пользователю выводится предупреждение о возможном повреждении данных. Это предотвращает потерю информации и упрощает диагностику ошибок.

Преимущества реализованных алгоритмов

– минимальные вычислительные затраты;

– простота восприятия и документирования;

– возможность расширения без изменения архитектуры;

– устойчивость к ошибкам ввода-вывода;

– прозрачность логики для проверки преподавателем.

Алгоритмы программы «Турагентство» построены по принципу модульности: каждый отвечает за строго определённую задачу, но может взаимодействовать с другими через общие структуры данных. Такое решение делает систему надёжной и лёгкой для сопровождения [5].

3 ПРОГРАММНАЯ РЕАЛИЗАЦИЯ

3.1 Алгоритмы

Реализация алгоритмов в программе «Турагентство» выполнена с использованием структурного и объектно-ориентированного подхода на языке C++ [1]. Каждый алгоритм представляет собой функцию или группу функций, отвечающих за определённую часть логики работы приложения.

В качестве основы выбрана модульная структура, которая включает:

- обработку запросов пользователей;
- работу с файлами данных;
- контроль целостности информации (хэш-суммы);
- обработку ошибок и некорректных вводов.

Для обеспечения читаемости и гибкости код разбит на отдельные файлы:

- main.cpp — основной файл с точкой входа;
- menu.cpp — функции отображения интерфейса;
- user.cpp, tour.cpp, booking.cpp — обработка пользователей, туров и бронирований соответственно.

Алгоритм регистрации пользователя

Алгоритм регистрации реализован в модуле `user.cpp` и позволяет добавить нового клиента или администратора в систему. Он включает следующие шаги:

1. Ввод имени пользователя и пароля , подтверждения пароля.
2. Проверка уникальности логина в файле `accounts.txt`.
3. При отсутствии совпадений — добавление новой строки с логином и паролем.
4. Сохранение изменений и сообщение об успешной регистрации.

```
=== Запущена программа туристического  агентства ===

=== Главное меню ===
1. Регистрация
2. Вход
3. Посмотреть туры
4. Забыл проль
0. Выход
Choice: 1

=== Регистрация ===
Логин: user
Пароль: 12345Qwe!

Пароль: 12345Qwe!
 Надежный пароль
Подтвердите пароль: 12345Qwe!
Регистрация завершена!

=== Главное меню ===
1. Регистрация
2. Вход
3. Посмотреть туры
4. Забыл проль
0. Выход
Choice:
```

Рисунок 3.1 – Регистрации пользователя

Алгоритм авторизации пользователя

Авторизация выполняется при входе в систему. Программа запрашивает логин и пароль, после чего сравнивает введённые данные с записями в `accounts.txt`. В случае совпадения данных открывается главное меню.

```
=== Главное меню ===
1. Регистрация
2. Вход
3. Посмотреть туры
4. Забыл проль
0. Выход
Choice: 2

=== Вход ===
Логин: user
Пароль: 12345Qwe!
Успешный вход!

=== Пользовательское меню ===
1. Забронировать тур
2. Отменить бронирование
3. Мои заказы
4. Поменять пароль
0. Выход из системы
Choice:
```

Рисунок 3.2 – Авторизация пользователя

Алгоритм добавления и редактирования туров

Добавление туров реализовано только для администратора. Алгоритм позволяет ввести параметры нового тура (страна, цена, длительность, описание), сохранить их в `tours.txt` и обновить список.

```
Введите название тура: Hot_tour_in_Rim
Введите страну: Italy
Введите цену: 500
Тур добавлен.
```

Рисунок 3.3 – Пример ввода данных о туре

```
Available tours:
1. Tour: Hot_tour_in_Rim | Country: Italy | Price: $500
```

Рисунок 3.4 – Отображение добавленного тура в общем списке

Алгоритм бронирования тура

При выборе пользователем пункта «Забронировать тур» программа выполняет следующие действия:

1. Выводит список доступных туров из `tours.txt`.
2. Предлагает пользователю выбрать нужный номер тура.
3. Проверяет наличие мест.
4. Создаёт запись в `bookings.txt` с логином, названием тура и датой.

```
=== Пользовательское меню ===
1. Забронировать тур
2. Отменить бронирование
3. Мои заказы
4. Поменять пароль
0. Выход из системы
Choice: 1

Available tours:
1. Tour: Hot_tour_in_Rim | Country: Italy | Price: $500
Введите номер тура для бронирования: 1
Бронирование прошло успешно.
```

Рисунок 3.5 – Окно консоли при бронировании тура

```
tours.txt
1  Hot_tour_in_Rim Italy 500
2
```

Рисунок 3.6 – Содержимое файла bookings.txt после бронирования

3.2 Функционал алгоритмов

Каждый реализованный алгоритм тесно связан с интерфейсом программы и отвечает за выполнение конкретных действий, доступных пользователю или администратору [2]. Для наглядности рассмотрим основные функциональные возможности.

Главное меню программы

Главное меню является центральной точкой взаимодействия пользователя с системой. Из него можно выбрать регистрацию, авторизацию, просмотр туров и выход.

```
=== Главное меню ===
1. Регистрация
2. Вход
3. Посмотреть туры
4. Забыл проль
0. Выход
```

Рисунок 3.7 – Главное меню программы “Турагентство”

Меню администратора

После успешной авторизации администратора открывается расширенное меню, включающее управление турами и пользователями.

```
=== Меню администратора ===
1. Добавить тур
2. Удалить тур
3. Показать туры
4. Показать все заказы
5. Обрабатывать запросы на сброс пароля
0. Выход из системы
```

Рисунок 3.8 – Меню администратора программы

Обработка ошибок и некорректных действий

Программа обрабатывает ситуации, когда пользователь вводит недопустимые значения (например, буквы вместо чисел). В таких случаях ввод очищается, а пользователю предлагается повторить действие.

```
=== Пользовательское меню ===
1. Забронировать тур
2. Отменить бронирование
3. Мои заказы
4. Поменять пароль
0. Выход из системы
Choice: 7
Invalid choice.
```

Рисунок 3.9 – Сообщение о некорректном вводе данных

РЕЗУЛЬТАТЫ И АНАЛИЗ

4.1 Результаты написания программы

Обзор программы

Реализованная программа предоставляет пользователю интуитивно понятный и функциональный инструмент для управления туристическими предложениями, бронирования туров и ведения учётных записей. Программа разработана в виде консольного приложения на языке C++, что обеспечивает её независимость от операционных систем и минимальные требования к ресурсам.

При запуске программы пользователь попадает в главное меню, где представлен набор базовых функций — регистрация, авторизация, просмотр доступных туров, бронирование и выход из системы. Интерфейс построен по принципу пошагового взаимодействия: на каждом этапе программа сопровождает пользователя подсказками и сообщениями, что делает работу максимально удобной и понятной.

Первое, что отмечает пользователь, — это логичная структура взаимодействия и мгновенная реакция программы. Все операции выполняются практически без задержек, что обеспечивается использованием стандартных контейнеров STL и эффективных алгоритмов обработки данных (std::vector, std::sort, std::getline). При выборе пунктов меню данные считываются из файлов, обрабатываются и выводятся в виде форматированной таблицы.

Особое внимание в реализации уделено обработке ошибок и проверке корректности данных. Если пользователь вводит недопустимые значения (например, буквы вместо числа), программа не завершается аварийно, а выводит сообщение об ошибке и предлагает повторить ввод. Такая логика делает систему устойчивой к ошибкам и повышает надёжность работы в реальных условиях эксплуатации.

Следующей важной возможностью является автоматическая проверка целостности данных. При запуске программы вычисляется контрольная хэш-сумма для каждого файла (accounts.txt, tours.txt, bookings.txt). Если хэш не совпадает с сохранённым значением, система уведомляет пользователя о возможном изменении или повреждении данных. Это решение повышает устойчивость программы к ошибкам при работе с файлами и делает хранение информации более надёжным.

Реализация функционала администратора заслуживает отдельного внимания. После авторизации под учётной записью администратора становятся доступны операции добавления и удаления туров, редактирования данных и просмотра бронирований. Это обеспечивает централизованное управление системой и имитирует логику работы реальных информационных систем туристических компаний.

Программа также реализует просмотр истории бронирований для каждого пользователя. Каждый заказ сохраняется в файле bookings.txt с указанием имени клиента, названия тура и даты бронирования. При просмотре истории данные считываются, фильтруются по имени и выводятся в читаемой форме, что демонстрирует применение фильтрации и поиска по критериям.

С точки зрения производительности программа показывает стабильные результаты. Все операции выполняются мгновенно даже при больших объёмах данных, что обусловлено линейной сложностью реализованных алгоритмов и отсутствием избыточных вычислений. Тестирование показало корректную работу всех функций, а также устойчивость к ошибочным действиям пользователя.

Результаты анализа работы программы подтверждают, что поставленные задачи полностью выполнены:

- реализована система учёта и хранения данных о турах, пользователях и бронированиях;
- обеспечена защита от ошибок ввода и случайных повреждений файлов;
- реализован контроль целостности данных;
- предусмотрен простой, понятный интерфейс, не требующий обучения пользователя;
- программа продемонстрировала высокую скорость выполнения операций и стабильную работу.

Таким образом, разработанное приложение является надёжным учебным программным продуктом, реализующим принципы структурного программирования и алгоритмизации. Программа успешно прошла тестирование и готова к использованию как демонстрационный пример при изучении дисциплин, связанных с разработкой программного обеспечения.

ЗАКЛЮЧЕНИЕ

В рамках данного курсового проекта было разработано программное обеспечение турагентства, предназначенное для автоматизации учёта туристических предложений, пользователей и бронирований. Реализованное решение создано на языке C++ с использованием стандартной библиотеки STL и демонстрирует высокую стабильность, простоту эксплуатации и корректную работу со всеми типами данных.

В процессе выполнения проекта были изучены основные подходы к организации хранения данных в текстовых файлах, принципы структурного программирования, алгоритмы сортировки, фильтрации и поиска информации , так же ООП. Кроме того, реализован механизм контроля целостности данных на основе вычисления хэш-суммы (CRC-подобный алгоритм), что позволило повысить надёжность хранения пользовательских данных.

Основным результатом работы является функциональная, устойчивая и расширяемая система, позволяющая эффективно автоматизировать базовые процессы работы туристического агентства. Программа имеет модульную структуру, что упрощает добавление новых функций — например, интеграцию с базой данных или создание графического интерфейса.

Дальнейшее развитие проекта может быть связано с внедрением современных СУБД (PostgreSQL, SQLite) для централизованного хранения информации, добавлением веб-интерфейса для удалённого доступа, а также интеграцией аналитических модулей для анализа продаж и предпочтений клиентов.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

[1] Лафорте, Р. Объектно-ориентированное программирование в С++ 4-е издание / Р. Лафорте. – СПб. : Питер, 2015. – 928 с.

[2] Страуструп, Б. Программирование. Принципы и практика с использованием С++ (2е издание) / Б. Страуструп. – СПб. : Питер, 2016. – 1328 с.

[3] Джарратано, Дж. Экспертные системы: принципы разработки и программирование (4е издание) / Дж. Джарратано, Гари Райли – Вильямс, 2007. – 1152 с.

[4] Язык программирования С лекции и упражнения: учеб.пособие / – Издательский дом "Вильямс" , 2013. – 960 с.

[5] Эккель, Б. Философия С++ (2 части) / Б. Эккель – Питер, 2004. – 572 с.

[6] Конспект лекций по курсу «Основы алгоритмизации и программирования» для студентов всех специальностей и всех форм обучения : – учеб. пособие / Т. М. Кривоносова. [и др.]. – Минск : БГУИР, 2004.– 89 с.

ПРИЛОЖЕНИЕ А
(ОБЯЗАТЕЛЬНОЕ)
ЛИСТИНГ ПРОГРАММНОГО КОДА

Листинг А.1 – main.cpp

```
#include <iostream>
#include <fstream>
#include <limits>
#include "Funcs.h"
#include "Utils.h"
#include "Crypto.h"
using namespace std;

// Меню пользователя
void userMenu(User& user, TourManager& tm, BookingManager& bm) {
    int choice;
    do {
        cout << endl << "=== Пользовательское меню ===" << endl;
        cout << "1. Забронировать тур" << endl;
        cout << "2. Отменить бронирование" << endl;
        cout << "3. Мои заказы" << endl;
        cout << "4. Поменять пароль" << endl;
        cout << "0. Выход из системы" << endl;
        cout << "Choice: ";
        if (!(cin >> choice)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            choice = -1;
        }

        switch (choice) {
            case 1: bm.makeBooking(user, tm, "bookings.txt"); break;
            case 2: bm.cancelBooking(user, "bookings.txt"); break;
            case 3: bm.showBookings(user); break;
            case 4: user.changeOwnPassword("accounts.txt"); break;
            case 0: user.logout(); break;
            default: cout << RED << "Invalid choice." << RESET << endl;
```

```
        }
    } while (choice != 0 && user.isLoggedIn());
}

// Меню администратора
void adminMenu(User& user, TourManager& tm, BookingManager& bm) {
    int choice;
    do {
        cout << endl << "=== Меню администратора ===" << endl;
        cout << "1. Добавить тур" << endl;
        cout << "2. Удалить тур" << endl;
        cout << "3. Показать туры" << endl;
        cout << "4. Показать все заказы" << endl;
        cout << "5. Обрабатывать запросы на сброс пароля" << endl;
        cout << "0. Выход из системы" << endl;
        cout << "Choice: ";
        if (!(cin >> choice)) {
            cin.clear();
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            choice = -1;
        }

        switch (choice) {
            case 1: tm.addTour("tours.txt"); break;
            case 2: tm.removeTour("tours.txt"); break;
            case 3: tm.showTours(); break;
            case 4: bm.viewAllBookings(); break;
                        case    5:    adminViewAndProcessRequests("requests.txt",
"accounts.txt"); break;
            case 0: user.logout(); break;
            default: cout << RED << "Invalid choice." << RESET << endl;
        }
    } while (choice != 0 && user.isLoggedIn());
}

int main() {

    //setlocale(LC_ALL, "Russian");
```

```
        cout << GREEN << "=== Запущена программа
туристического агентства ===" << RESET << endl;

    // Автоматическая миграция паролей при запуске
    SimpleCrypto::autoMigratePasswords("accounts.txt");
    SimpleCrypto::autoMigratePasswords("requests.txt");

    // Проверка наличия администратора
    ifstream check("accounts.txt");//ifstream поток для чтения из файла
    bool adminFound = false;
    if (check.is_open()) {
        string u, p;
          while (check >> u >> p) {//операция извлечения (operator>>) из
потока.
            if (u == "admin") { adminFound = true; break; }
        }
        check.close();
    }
    if (!adminFound) {
        ofstream create("accounts.txt", ios::app);
        if (create.is_open()) {
            // Хэшируем пароль администратора по умолчанию
                string hashedPassword = SimpleCrypto::hashPassword("admin",
"admin");
            create << "admin " << hashedPassword << endl;
            create.close();
        }
    }

    User user;
    TourManager tourManager;
    tourManager.loadTours("tours.txt");

    BookingManager bookingManager;
    bookingManager.loadBookings("bookings.txt");

    int choice;
```

```
do {
    cout << endl << "=== Главное меню ===" << endl;
    cout << BLUE << "1. Регистрация" << RESET << endl;
    cout << GREEN << "2. Вход" << RESET << endl;
    cout << "3. Посмотреть туры" << endl;
    cout << YELLOW << "4. Забыл проль " << RESET << endl;
    cout << RED << "0. Выход" << RESET << endl;
    cout << "Choice: ";
    if (!(cin >> choice)) {
        cin.clear();
        cin.ignore(numeric_limits<streamsize>::max(), '\n');
        choice = -1;
    }

    switch (choice) {
        case 1:
            user.registerAccount("accounts.txt");
            break;
        case 2:
            user.login("accounts.txt");
            if (user.isLoggedIn()) {
                tourManager.loadTours("tours.txt");
                bookingManager.loadBookings("bookings.txt");
                if (user.isAdmin()) adminMenu(user, tourManager,
bookingManager);
                else userMenu(user, tourManager, bookingManager);
            }
            break;
        case 3:
            tourManager.showTours();
            break;
        case 4:
            requestPasswordReset("requests.txt");
            break;
        case 0:
            cout << RED << "Exiting program." << RESET << endl;
            break;
        default:
```

```
            cout << RED << "Invalid choice." << RESET << endl;
        }
    } while (choice != 0);

    return 0;
}
```

Листинг А.2 – Crypto.h

```
#pragma once
#include <string>
#include <sstream>
#include <iomanip>
#include <vector>
#include <fstream>
using namespace std;

// Простое хэширование паролей
// Это базовый метод для учебных целей (в реальных проектах используйте bcrypt/SHA-256)
class SimpleCrypto {
private:
    // Простая хэш-функция для паролей
    static unsigned int simpleHash(const string& str) {
        unsigned int hash = 5381;
        for (char c : str) {
            hash = ((hash << 5) + hash) + (unsigned char)c; // hash * 33 + c
        }
        return hash;
    }

public:
    // Хэширование пароля (логин + пароль)
    static string hashPassword(const string& login, const string& password) {
        string combined = login + password;
        unsigned int hash = simpleHash(combined);

        // Преобразуем в строку
```

```
        stringstream ss;
        ss << hash;
        return ss.str();
    }

    // Проверка пароля
     static bool verifyPassword(const string& login, const string& password,
const string& storedHash) {
        return hashPassword(login, password) == storedHash;
    }

    // Автоматическая миграция паролей из старого формата в хэш
    static void autoMigratePasswords(const string& filename) {
        ifstream in(filename);
        if (!in.is_open()) return;
        vector<pair<string, string>> users; // login, password_or_hash
        string login, passData;
        bool needsMigration = false;
        while (in >> login >> passData) {
            // Проверяем, является ли это хэшем:
              // Хэш - это строка из цифр длиной 9-11 символов (результат
simpleHash)
            bool isHash = true;
            // Проверка 1: Длина хэша обычно 9-11 символов
            if (passData.length() < 9 || passData.length() > 11) {
                isHash = false;
            } else {
                // Проверка 2: Все символы должны быть цифрами
                for (char c : passData) {
                    if (!isdigit(c)) {
                        isHash = false;
                        break;
                    }
                }
            }

                // Дополнительная проверка: если это выглядит как plaintext
пароль
```

```
            if (passData.length() <= 20 && !isHash) {
                // Скорее всего это plaintext пароль
                needsMigration = true;
            }
            users.push_back({login, passData});
        }
        in.close();
        if (needsMigration) {
            ofstream out(filename + ".migrated");
            for (const auto& user : users) {
                // Проверяем каждого пользователя отдельно
                bool isHash = true;

                if (user.second.length() < 9 || user.second.length() > 11) {
                    isHash = false;
                } else {
                    for (char c : user.second) {
                        if (!isdigit(c)) {
                            isHash = false;
                            break;
                        }
                    }
                }
                if (!isHash) {
                    // Хэшируем старый plaintext пароль
                    string hash = hashPassword(user.first, user.second);
                    out << user.first << " " << hash << endl;
                } else {
                    // Уже хэш - оставляем как есть
                    out << user.first << " " << user.second << endl;
                }
            }
            out.close();
            remove(filename.c_str());
            rename((filename + ".migrated").c_str(), filename.c_str());
    }
    }
};
```

ПРИЛОЖЕНИЕ Б
(ОБЯЗАТЕЛЬНОЕ)
БЛОК СХЕМА АЛГОРИТМА

ПРИЛОЖЕНИЕ В
(ОБЯЗАТЕЛЬНОЕ)
ПЛАКАТ ПРОГРАММЫ

ПРИЛОЖЕНИЕ Г
(ОБЯЗАТЕЛЬНОЕ)
ВЕДОМОСТЬ ДОКУМЕНТОВ